Чудинова Анна

https://colab.research.google.com/drive/14VHUlTWiopWKcG79gSMqmW-YuwDt-cMF?usp=sharing

---

1. clients

Представьте, что вы устроились работать Дата-инженером в некоторый интернет магазин. До вас в этой компании уже работал один разработчик, который придумал небольшую базу данных для этого магазина и потом неожиданно уволился. Ваша задача провести проверку существующей архитектуры и решить корректна ли она или можно внести некоторые доработки. Вы обратили внимание на таблицу с информацией о зарегистрировавшихся покупателях (клиентах).

Таблица имеет следующую схему:

```
clients (
  client_id number,             --уникальный id клиента
  client_name varchar(255),     --имя клиента
  client_surname varchar(255),  --фамилия клиента
  login varchar(30),            --логин, который придумал клиент
  city_id number,               --id города, который указал клиент
  (в интерфейсе выбирается название города, а в таблицу сохраняется id)
  age number,                   --возраст клиента
  reg_date date                 –-дата регистрации на сайте
);
```

Считаете ли вы данный набор и смысл полей корректным?

Ответ

Я бы предложила немного изменить таблицу.

Для начала необходимо:

- столбец с возрастом клиента age заменить на столбец с информацией о дате рождения клиента. Так, со временем актуальность данных не пропадет;
- также стоит добавить колонку с номером телефона или email для связи с клиентом или рассылки информации. Эта же информация может пригодиться для сброса пароля.

Дополнительно можно улучшить:

- добавить атрибут PRIMARY KEY к столбцу с уникальными id клиентов. Так, мы гарантируем уникальность id клиента;
- добавить атрибут UNIQUE колонкам, поля которых также должны быть уникальными – login, email. Причем можно опционально добавить NOT NULL, если необходимо, чтобы поле обязательно было заполнено.

Таблица имеет следующую схему:

```
clients (
    client_id number PRIMARY KEY,         -- уникальный id клиента
    client_name varchar(255),               -- имя клиента
    client_surname varchar(255),             -- фамилия клиента
    login varchar(30) UNIQUE NOT NULL,    -- уникальный логин, который придумал клиент
    email varchar(255) UNIQUE NOT NULL    -- уникальный email
    city_id number,                           -- id города, который указал клиент
    (в интерфейсе выбирается название города, а в таблицу сохраняется id)
    birth_date date,                           -- возраст клиента
    reg_date date                               -- дата регистрации на сайте
);
```

---

2. SCD

Представьте, что вы устроились работать Дата-инженером в некоторый интернет магазин. До вас в этой компании уже работал один разработчик, который придумал небольшую базу данных для этого магазина и потом неожиданно уволился. Ваша задача провести проверку существующей архитектуры и решить корректна ли она или можно внести некоторые доработки. Вы обратили внимание на таблицу с информацией о товарах магазина. Таблица имеет следующую схему:

```
items (
  item_id number, --уникальный id товара
  item_name varchar(255), --наименование товара
  item_cost number –стоимость товара
);
```

Глядя на эту таблицу вы вспоминаете, что мы живём в рыночной экономике и цена, если не каждый день, то каждый месяц может меняться. К примеру, сезонные повышения цен или наоборот скидки (перед началом учебного года или новым годом). Так же вы знаете что у магазина есть отдел финансовой отчётности, в котором сотрудникам нужно строить различны отчёты (о доходах например) за разные промежутки времени, в том числе и за прошлые месяцы и даже прошлые года.

Как бы вы доработали архитектуру таблицы, что бы обеспечить историческое хранение стоимости товара?

Ответ

В первую очередь необходимо обеспечить историчность данных. Для этого используем метод SCD2 и создадим отдельную таблицу с ценами, а всю информацию о товаре (***item_name***) выделим в отдельную таблицу.

Таблицы будут выглядеть следующим образом:

- для активных записей в столбце ***item_cost_date_to*** цена будет установлена по умолчанию, например, ***01.01.9999***
- для удобства можно добавить столбец ***is_active*** для обозначения актуальных цен, что может облегчить поиск нужной информации

```
items_costs (
  id number PRIMARY KEY,          -- уникальный id записи
  item_id number,                     -- уникальный id товара
  item_cost number,                    -- стоимость товара
  start_dt date,                     -- дата установки стоимости товара
  end_dt date default '01.01.9999',     -- дата, до которой актуальна стоимость товара
  is_active                       -- '1' - если запись актуальна, иначе 0
);

items (
  id number PRIMARY KEY, -- уникальный id товара
  item_name varchar(255),     -- наименование товара
…
);
```

Также, в задаче упоминаются сезонные акции. Для записи информации об акциях можно добавить соответствующую таблицу, а в ***items_costs*** добавить столбец ***ivent_id***:

```
items_costs (
  id number PRIMARY KEY, -- уникальный id записи
  item_id number,         -- уникальный id товара
  ivent_id number,         -- уникальный id акции или NULL, если цена установлена вне акции
  item_cost number,   -- стоимость товара
  start_dt date,         -- дата установки стоимости товара
  end_dt date end_dt,    -- дата, до которой актуальна стоимость товара
  is_active           -- '1' - если запись актуальна, иначе 0
);

items (
  id number PRIMARY KEY,     -- уникальный id товара
  item_name varchar(255),     -- наименование товара
…
);

ivents (
  id number PRIMARY KEY,       -- уникальный id акции
  ivent_name varchar(255),     -- наименование акции
  start_dt date,             -- дата начала акции
  end_dt date,               -- последний день акции
  info varchar(255),           -- краткая информация
);
```

Ошибки в коде

Требуется проверить запрос на корректность и исправить там ошибки, если они есть.

Существует учебная схема HR, содержащая таблицы: employees, departments и locations. Необходимо получить все отделы0, расположенные в Сеуле в которых все сотрудники, не имеющие менеджера, зарабатывают в общей сложности более 100000. Для решения задачи был написан следующий запрос:

```
 SELECT DEPARTMENT_ID,
     DEPARTMENT_NAME,
     SUM(SALARY) TOTAL_SALARY
 FROM EMPLOYEE E,
 DEPARTMENTS D
 WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID
   AND MANAGER_ID = NULL
   AND LOCATION_ID = (SELECT LOCATION_ID
                      FROM LOCATIONS
                      WHERE CITY = 'SEOUL')
   AND SUM(SALARY) >= 100000
 GROUP BY DEPARTMENT_NAME
```

Ответ

- добавила **left join**
- везде для точного определения колонок прописала их принадлежность к таблицам:
  - предполагаю, что информация о названии отдела и **location_id** - в таблице **departments**
  - информация и зарплатах и наличии менеджера у сотрудников - в **employees** employees, departments и locations
- MANAGER_ID **IS NULL** вместо **=**
- исправила ошибку в названии таблицы **employees**
- LOCATION_ID **IN** (подзапрос)
- прописала все столбцы, идущие в SELECT до аггрегации, в **GROUP BY**
- после group by данные фильтруются с помощью **HAVING**
- так как по условию необходимы отделы с суммарной зарплатой определенных сотрудников **больше** 100000, то поставила оператор **>**

```
SELECT E.DEPARTMENT_ID,
       D.DEPARTMENT_NAME,
       SUM(E.SALARY) TOTAL_SALARY
FROM EMPLOYEES E
LEFT JOIN DEPARTMENTS D ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
WHERE E.MANAGER_ID IS NULL
  AND D.LOCATION_ID IN (SELECT LOCATION_ID
                        FROM LOCATIONS
                        WHERE CITY = 'SEOUL')
GROUP BY E.DEPARTMENT_ID,
         D.DEPARTMENT_NAME
HAVING SUM(E.SALARY) > 100000;
```

Карандаш

В некоторой комнате на пол уронили карандаш. Объясните почему вы не можете через него перепрыгнуть?

Ответ

- Я нахожусь в другой комнате или даже в здании напротив таким образом, что вижу карандаш, но добраться до него не могу.

Или:

- Карандаш на самом деле представляет собой какой-то огромный объект (скульптуру), который я не могу физически перепрыгнуть.